

Graph and Knapsack Project Documentation

Fitsum Gelaye

December 7, 2024

Contents

1	Introduction	2
2	Graph Representation	2
2.1	Edge Class	2
2.2	Graph Class	2
2.3	Graph Parsing Function	3
3	Bellman-Ford Algorithm	4
4	Spice Structure	6
4.1	Code	6
4.2	Description	6
5	Input Parsing	6
5.1	Code	6
5.2	Description	7
6	Fractional Knapsack Algorithm	7
6.1	Code	8
6.2	Description	8
7	Results and Discussion	8
7.1	Graph Analysis	8
7.2	Knapsack Analysis	9
7.3	Single-Source Shortest Path (SSSP) using Bellman-Ford Algorithm . . .	9
7.3.1	Bellman Ford Description	9
7.3.2	Running Time Derivation	10
7.4	Running Time Analysis for Fractional Knapsack	10

1 Introduction

This project implements two main functionalities:

- Shortest Path calculation using the Bellman-Ford algorithm.
- Fractional Knapsack problem for optimizing the selection of items.

It processes graph data and spice data from input files, performing calculations and outputting results.

2 Graph Representation

The graph is implemented using an `Edge` class and a `Graph` class. The Bellman-Ford algorithm calculates the shortest path from a source vertex to all other vertices.

2.1 Edge Class

```
1 #ifndef EDGE_H
2 #define EDGE_H
3
4 class Edge {
5 public:
6     int start, end, weight;
7     Edge(int s, int e, int w) : start(s), end(e), weight(w) {}
8 };
9
10 #endif
```

Listing 1: Edge Class Definition

Code Description:

- **Lines 4–9:** The `Edge` class encapsulates the structure of an edge, with three public members: `start`, `end`, and `weight`.
- **Line 8:** Constructor initializes the edge's starting vertex, ending vertex, and weight.

2.2 Graph Class

```
1 class Graph {
2 private:
3     int numVertices;
4     vector<Edge> edges;
5
6 public:
7     Graph(int vertices) : numVertices(vertices) {}
8
9     void addEdge(int start, int end, int weight) {
10         edges.push_back(Edge(start, end, weight));
11     }
12 }
```

```

12
13     vector<Edge>& getEdges() {
14         return edges;
15     }
16
17     int getNumVertices() {
18         return numVertices;
19     }
20 };

```

Listing 2: Graph Class Definition

Code Description:

- **Lines 1–4:** Declare private members, including the number of vertices `numVertices` and a vector of edges (`edges`).
- **Lines 6–7:** Constructor initializes the graph with the specified number of vertices.
- **Lines 9–11:** `addEdge` method adds an edge to the graph by constructing an `Edge` object and attaching it to the `edges` vector.
- **Lines 13–14:** `getEdges` returns the list of edges.
- **Lines 16–17:** `getNumVertices` retrieves the total number of vertices.

2.3 Graph Parsing Function

```

1  Graph parseGraph(const string& inputFile) {
2      ifstream file(inputFile);
3      if (!file.is_open()) {
4          cerr << "Error: Could not open file" << inputFile << endl;
5          exit(1);
6      }
7
8      string line;
9      Graph* graph = nullptr;
10     unordered_map<int, bool> vertexSet;
11
12     while (getline(file, line)) {
13         istringstream iss(line);
14         string command;
15         iss >> command;
16
17         if (command == "new" && iss >> command && command == "graph") {
18             continue;
19         } else if (command == "add") {
20             string type;
21             iss >> type;
22
23             if (type == "vertex") {
24                 int vertex;
25                 iss >> vertex;
26                 vertexSet[vertex] = true;
27             } else if (type == "edge") {

```

```

28         int start, end, weight;
29         char dash;
30         if (iss >> start >> dash >> end >> weight && dash == '-')
31             {
32                 if (!graph) {
33                     graph = new Graph(vertexSet.size());
34                 }
35                 graph->addEdge(start, end, weight);
36             } else {
37                 cerr << "Error: Invalid edge format." << endl;
38                 exit(1);
39             }
40         }
41     }
42
43     if (!graph) {
44         cerr << "Error: No graph was created." << endl;
45         exit(1);
46     }
47     return *graph;
48 }

```

Listing 3: Graph Parsing Function

Code Description:

- **Lines 1–6:** Open the input file and check if it was successfully opened.
- **Lines 8–10:** Declare a `Graph` pointer and an `unordered_map` to track unique vertices.
- **Lines 12–31:** Read each line from the input file:
 - If the command is "new graph", skip further processing for the line.
 - If the command is "add vertex", extract the vertex number and add it to the vertex set.
 - If the command is "add edge", extract edge details, initialize the graph if not already done, and add the edge.
- **Lines 31–35:** If no graph was created by the end of file parsing, display an error and terminate.
- **Line 36:** Return the created `Graph` object by dereferencing the pointer.

3 Bellman-Ford Algorithm

```

1 void bellmanFord(Graph& graph, int source) {
2     int V = graph.getNumVertices();
3     vector<Edge>& edges = graph.getEdges();
4     vector<int> distance(V + 1, numeric_limits<int>::max());
5     vector<int> predecessor(V + 1, -1);
6

```

```

7     distance[source] = 0;
8
9     for (int i = 1; i <= V - 1; ++i) {
10         for (const Edge& edge : edges) {
11             if (distance[edge.start] != numeric_limits<int>::max() &&
12                 distance[edge.start] + edge.weight < distance[edge.end]
13                 ) {
14                 distance[edge.end] = distance[edge.start] + edge.weight
15                 ;
16                 predecessor[edge.end] = edge.start;
17             }
18         }
19     }
20
21     for (const Edge& edge : edges) {
22         if (distance[edge.start] != numeric_limits<int>::max() &&
23             distance[edge.start] + edge.weight < distance[edge.end]) {
24             cout << "Graph contains a negative weight cycle." << endl;
25             break;
26         }
27     }
28
29     for (int i = 1; i <= V; ++i) {
30         if (distance[i] == numeric_limits<int>::max()) {
31             cout << "Vertex " << i << " is not reachable from vertex "
32                 << source << "." << endl;
33         } else {
34             cout << "1-->" << i << " cost is " << distance[i] << ";
35             path:
36             vector<int> path;
37             for (int v = i; v != -1; v = predecessor[v]) {
38                 path.insert(path.begin(), v);
39             }
40
41             for (size_t j = 0; j < path.size(); ++j) {
42                 cout << path[j];
43                 if (j != path.size() - 1) cout << "-->";
44             }
45             cout << endl;
46         }
47     }
48 }

```

Listing 4: Bellman-Ford Algorithm

Code Description:

- **Lines 1–5:** Initialize variables for the number of vertices, edge list, distances, and predecessors.
- **Line 7:** Set the distance to the source vertex as zero.
- **Lines 9–15:** Relax edges up to $V-1$ times. Update the distance and predecessor for each vertex if a shorter path is found.
- **Lines 17–21:** Detect negative weight cycles by checking if further relaxation is possible.

- **Lines 23–37:** Print the shortest distance and path for each vertex. Construct the path using the predecessor array.

4 Spice Structure

The ‘Spice’ struct represents the details of a single item, including its name, quantity, total price, and derived value per unit.

4.1 Code

```

1 struct Spice {
2     string name;
3     int quantity;
4     double total_price;
5     double value_per_unit;
6
7     Spice(string n, double tp, int q) : name(n), total_price(tp),
8         quantity(q) {
9         value_per_unit = total_price / quantity;
10    }
11 };

```

Listing 5: Spice Structure Definition

4.2 Description

- **Lines 1–2:** Declare a structure to store details about spices.
- **Lines 3–4:** Define member variables for the spice’s name, quantity, total price, and value per unit.
- **Lines 6–8:** Constructor initializes the ‘Spice’ object with the provided name, total price, and quantity. It also calculates the ‘value_per_unit’ by dividing the total price by the quantity.

5 Input Parsing

The ‘parseInput’ function reads the spice details and knapsack capacities from an input file. The function expects a specific format where spice details and knapsack capacities are defined on separate lines.

5.1 Code

```

1 void parseInput(const string& filename, vector<Spice>& spices, vector<
2     int>& knapsack_capacities) {
3     ifstream file(filename);
4     if (!file.is_open()) {
5         cerr << "Error: Could not open file " << filename << endl;
6         exit(1);
7     }
8 }

```

```

7
8     string line;
9     while (getline(file, line)) {
10         istringstream iss(line);
11
12         // Parse spice details
13         if (line.find("spice_name") != string::npos) {
14             string name;
15             double total_price;
16             int quantity;
17             iss.ignore(15, '='); // Skip "spice name ="
18             iss >> name;
19             iss.ignore(20, '='); // Skip "total_price ="
20             iss >> total_price;
21             iss.ignore(10, '='); // Skip "quantity ="
22             iss >> quantity;
23             spices.emplace_back(name, total_price, quantity);
24         }
25
26         // Parse knapsack capacities
27         if (line.find("knapsack_capacity") != string::npos) {
28             int capacity;
29             iss.ignore(20, '='); // Skip "knapsack capacity ="
30             iss >> capacity;
31             knapsack_capacities.push_back(capacity);
32         }
33     }
34     file.close();
35 }

```

Listing 6: Input Parsing Function

5.2 Description

- **Lines 1–4:** Open the input file for reading. If the file cannot be opened, an error message is displayed and the program exits.
- **Lines 6–24:** Loop through each line in the file:
 - If the line contains "spice name", parse the name, total price, and quantity of a spice.
 - Add the parsed spice to the 'spices' vector.
- **Lines 25–29:** If the line contains "knapsack capacity", parse the capacity and add it to the 'knapsack_capacities' vector.
- **Line 31:** Close the file.

6 Fractional Knapsack Algorithm

The 'fractionalKnapsack' function distributes spices into knapsacks to maximize total value. It selects items based on their value per unit, filling each knapsack until its capacity is reached.

6.1 Code

```
1 void fractionalKnapsack(const vector<Spice>& spices, vector<int>&
   knapsacks) {
2     vector<Spice> sorted_spices = spices;
3     // Sort spices by value per unit in descending order
4     sort(sorted_spices.begin(), sorted_spices.end(), [](const Spice& a,
       const Spice& b) {
5         return a.value_per_unit > b.value_per_unit;
6     });
7
8     for (int capacity : knapsacks) {
9         double total_value = 0.0;
10        cout << "Knapsack capacity: " << capacity << ":\n";
11        for (const Spice& spice : sorted_spices) {
12            if (capacity == 0) break;
13
14            int amnt_took = min(capacity, spice.quantity);
15            double val_taken = amnt_took * spice.value_per_unit;
16            total_value += val_taken;
17            capacity -= amnt_took;
18
19            cout << "    Took " << amnt_took << " of " << spice.name
20                << " (value: " << fixed << setprecision(2) <<
21                val_taken << ")\n";
22        }
23        cout << "Total value in knapsack: " << fixed << setprecision(2)
24            << total_value << "\n\n";
25    }
26 }
```

Listing 7: Fractional Knapsack Algorithm

6.2 Description

- **Lines 1–3:** Copy the ‘spices’ vector into ‘sorted_spices’ and sort it in descending order by ‘value_per_unit’.
- **Lines 5–17:** For each knapsack:
 - Loop through the sorted spices and take as much as possible without exceeding the knapsack’s capacity.
 - Update the total value of the knapsack based on the amount of spice taken.
 - Print details about the amount of spice taken and the corresponding value.
- **Lines 18–19:** Print the total value of all spices placed in the current knapsack.

7 Results and Discussion

7.1 Graph Analysis

Running the Bellman-Ford algorithm on the CLRS and class example graph from the input file produced the following paths and costs:

```
1 --> 1 cost is 0; path: 1
1 --> 2 cost is 2; path: 1 --> 4 --> 3 --> 2
1 --> 3 cost is 4; path: 1 --> 4 --> 3
1 --> 4 cost is 7; path: 1 --> 4
1 --> 5 cost is -2; path: 1 --> 4 --> 3 --> 2 --> 5
```

7.2 Knapsack Analysis

The Fractional Knapsack algorithm produced the following results:

```
Knapsack capacity: 1:
Took 1 of orange; (value: 9.00)
Total value in knapsack: 9.00
```

```
Knapsack capacity: 6:
Took 2 of orange; (value: 18.00)
Took 4 of blue; (value: 20.00)
Total value in knapsack: 38.00
```

```
Knapsack capacity: 10:
Took 2 of orange; (value: 18.00)
Took 8 of blue; (value: 40.00)
Total value in knapsack: 58.00
```

```
Knapsack capacity: 20:
Took 2 of orange; (value: 18.00)
Took 8 of blue; (value: 40.00)
Took 6 of green; (value: 12.00)
Took 4 of red; (value: 4.00)
Total value in knapsack: 74.00
```

```
Knapsack capacity: 21:
Took 2 of orange; (value: 18.00)
Took 8 of blue; (value: 40.00)
Took 6 of green; (value: 12.00)
Took 4 of red; (value: 4.00)
Total value in knapsack: 74.00
```

The algorithm successfully maximized the total value in each knapsack by selecting items with the highest value per unit.

7.3 Single-Source Shortest Path (SSSP) using Bellman-Ford Algorithm

7.3.1 Bellman Ford Description

The Bellman-Ford algorithm works as follows:

1. **Initialization:** Set the distance of the source vertex to 0 and all other vertices to infinity.
2. **Edge Relaxation:** Repeat $V - 1$ times (where V is the number of vertices), and for each edge (u, v) with weight w , update the distance to vertex v if the distance to $u + w$ is less than the current distance to v .
3. **Negative Cycle Detection:** After $V - 1$ iterations, check all edges again. If any distance can still be updated, there is a negative-weight cycle.

7.3.2 Running Time Derivation

The Bellman-Ford algorithm's time complexity is $O(V \cdot E)$, where:

- V is the number of vertices in the graph.
- E is the number of edges in the graph.

7.4 Running Time Analysis for Fractional Knapsack

- **Sorting:** The spices are sorted by their value per unit, which takes $O(N \log N)$, where N is the number of spices.
- **Knapsack Filling:** For each knapsack, the algorithm iterates through the spices and performs constant-time operations for each spice. This step takes $O(M \cdot N)$, where M is the number of knapsacks.
- **Overall Time Complexity:** The overall time complexity is $O(N \log N + M \cdot N)$.

This complexity arises because the algorithm must sort the items before iterating through them to select the most valuable items.